

Algoritmi e Strutture Dati

Appello del 20/2/2026 - Prova di programmazione (12 punti)

1. (3 punti)

Sono dati il quasi ADT Item, il tipo Key e le funzioni KEYcmp e ITEMprint, definiti sotto:

```
#define MAXS 40
typedef struct {
    char name[MAXS];
    float val;
} Item;
typedef char *Key;
int KEYcmp(Key k1, Key k2) {
    return strcmp(k1,k2);
}
void ITEMprint(Item *item) {
    printf("name: %s, val: %f\n", item->name, item->val);
}
```

Sapendo che il campo name è la chiave, si scriva la funzione KEYget, che, dato un Item, ne ritorni la chiave.

Si realizzi poi la funzione ProcessItems che, dato un vettore di Item, ritorni 3 risultati

- una stringa dinamica contenente la concatenazione delle chiavi presenti nel vettore (la stringa va allocata esattamente della dimensione necessaria, senza sovrallocazione),
- il puntatore all'item contenente la chiave più grande (usando KEYcmp per i confronti)
- la somma di tutti i campi val nel vettore.

La funzione deve poter essere chiamata da un main come il seguente.

```
int main (void) {
    Item vet[]={{"Parigi",2.2},{ "Roma",2.75},{ "Londra",8.8},{ "Berlino",3.75}};
    Item *maxp;
    char *keys; float sumVal;
    int n=sizeof(vet)/sizeof(Item);
    sumVal = ProcessItems(vet,n,&keys,&maxp);
    printf("La somma dei valori e': %f\n", sumVal);
    printf("La concatenazione delle stringhe e': %s\n", keys);
    printf("Il massimo e': "); ITEMprint(maxp);
    free(keys);
    return 0;
}
```

Con il main riportato, l'output sarebbe:

La somma dei valori e': 17.500000

La concatenazione delle stringhe e': ParigiRomaLondraBerlino

Il massimo e': name: Roma, val: 2.750000

```
Key KEYget(Item *item) { // E' necessario il puntatore ad Item
    return item->name;
}

float ProcessItems(Item *vet, int n, char **keys_p, Item **maxpp) {
    int totLen = 0;
    float sum=0.0;
    char *keys;
    Item *maxp = NULL;
    for (int i=0; i<n; i++) {
        sum += vet[i].val; // calcola somma
        totlen += strlen(vet[i].name); // lunghezza totale delle stringhe
        if (maxp==NULL || KEYcmp(KEYget(&vet[i]),KEYget(maxp))>0)
            maxp = &vet[i]; // massimo
    }
    *keys_p = malloc(totLen+1);
    for (int i=0; i<n; i++)
        strcat(*keys_p, vet[i].name);
    *maxpp = maxp;
    return sum;
}
```

```

}
keys = malloc(totLen*sizeof(char)+1); // aggiungi 1 per '\0'
keys[0]='\0'; // inizializza con stringa vuota
for (int i=0; i<n; i++)
    strcat(keys,vet[i].name); // concatena stringhe

*keys_p = keys;
*maxpp = maxp;
return sum;
}

```

2. (4 punti)

È dato un tipo BST (ADT di prima classe), avente nodi estesi in modo tale da contenere puntatore al padre e dimensione del sottoalbero. Il tipo Item (quasi ADT) e le struct BSTnode (con puntatore link) e binarysearchtree sono definiti come segue

```

typedef struct {
    char name[MAXC];
} Item;
typedef struct BSTnode* link;
struct BSTnode { Item item; link p; link l; link r; int N; };
struct binarysearchtree { link root; link z; };

```

Si scriva una funzione che, dato un BST b, generi un duplicato (un BST con la stessa topologia, contenente un duplicato dei dati).

La funzione abbia prototipo

BST BSTdup(BST b);

Se utile per la soluzione, si possono richiamare funzioni spiegate a lezione.

```

static link dupR(link h, link z, link p, link z2) {
    if (h==z) {
        return z2;
    }

    link l= dupR(h->l, z, h, z2);
    link r = dupR(h->r, z, h, z2);
    return NEW(h->item, p, l, r, l->N+r->N+1);
}

BST BSTdup(BST b) {
    BST b2 = BSTinit();
    b2->root = dupR(b->root, b->z, b2->z, b2->z);
    return b2;
}

```

3. (5 punti)

Si vogliono contare i numeri in una data base (base, compresa tra 2 e 10), aventi un dato numero di cifre (nDigits) tali da rispettare i seguenti vincoli:

- D. ogni cifra può comparire al più maxRip volte nel numero
- E. la differenza (in valore assoluto) tra due cifre consecutive può essere al massimo 2

F. il numero di cifre distinte (senza tener conto delle ripetizioni) deve essere almeno pari a $nDigits/2$ (divisione tra interi)

Ad esempio, con base 5, 7 cifre, massimo numero di ripetizioni 3, il numero 5356533 rispetta i vincoli, 3232224 non rispetta il primo vincolo, 5376551 non rispetta il secondo, 3223322 non rispetta il primo e il terzo.

La funzione da realizzare ritorna come risultato il conteggio dei numeri che rispettano i vincoli. Il prototipo è

```
int cntNum(int base, int nDigits, int maxRip);
```

```
/*
la soluzione e' una variante del modello delle disposizioni con ripetizioni,
in cui il numero di ripetizioni e' limitato a maxRip. SI usa a tale scopo
il vettore mark. Il vettore val non viene usato in quanto le cifre numeriche
conciderebbero con gli indici.
Si fa pruning sulla differenza tra cifre adiacenti. Il conteggio delle cifre distinte
viene aggiornato ad ogni chiamata ricorsiva e verificato nel caso terminale.
*/
int disp_rip(int pos, int *sol, int *mark, int n, int k, int m, int cntDist){
    int i, cnt=0;
    if (pos >= k)
        return (cntDist>k/2)? 1 : 0;

    for (i=0; i<n; i++){
        if ((mark[i] < m) &&
            (pos==0 || abs(i-sol[pos-1])<3)) { // pruning
            if (mark[i]++ == 0) cntDist++; // conta cifre distinte
            sol[pos] = i;
            cnt += disp_rip(pos+1, sol, mark, n, k, m, cntDist);
            if (--mark[i]==0) cntDist--;
        }
    }
    return cnt;
}

int cntNum(int base, int nDigits, int maxRip) {
    int *sol=malloc(nDigits*sizeof(int));
    int *mark=calloc(base,sizeof(int));
    int nSol = disp_rip(0,sol,mark,base,nDigits,maxRip,0);
    free(sol);
    free(mark);
    return nSol;
}
```

Algoritmi e Strutture Dati

Appello del 20/2/2026 - Prova di programmazione (18 punti)

Descrizione del problema

È dato un mazzo da 52 carte, in cui per ognuno dei 4 semi (cuori, quadri, fiori e picche) sono presenti 13 carte di 13 valori: carte numeriche da 1 a 10, più 3 figure (fante, donna e re). In questo contesto, per semplicità, si può supporre di attribuire alle tre figure i valori numerici 11 (fante), 12 (donna), 13 (re).

Dal mazzo viene selezionato un insieme S di carte, di cui si ha l'elenco (una lista concatenata).

Si vuole realizzare un'opportuna struttura dati, adatta a risolvere un problema di verifica e uno di ottimizzazione, come descritto nel seguito.

Attenzione: l'efficienza/complessità delle funzioni realizzate sarà oggetto di valutazione.

Strutture dati

Si definisca un adeguato tipo **Card** in grado di rappresentare, come quasi ADT, una carta da gioco: una carta deve essere caratterizzata da una stringa (il nome per esteso: es. otto, fante, ...), il seme (codificato mediante un tipo enum), il valore (un numero da 1 a 13).

Si definiscano i tipi necessari per realizzare come ADT di prima classe una lista concatenata di carte: sia **CardLIST** il nome del tipo ADT. Il tipo ADT viene usato per rappresentare sia un insieme che una sequenza di carte. Per eventuali operazioni sulle liste, se lo si ritiene opportuno, è possibile utilizzare (scrivendone unicamente il prototipo) funzioni (standard) spiegate a lezione. Se sono necessarie altre funzioni, vanno realizzate.

Si definisca infine un tipo ADT di prima classe **SOL**, in grado di contenere due liste di carte che rappresentino il risultato del problema di ottimizzazione

```
// cards.h
typedef struct cardlist *CardLIST;
typedef struct sol *SOL;
typedef enum {cuori, quadri, fiori, picche} seed_e;

typedef struct {
    char name[MAXS];
    seed_e seed;
    int val;
} Card;

// cards.c
typedef struct node *link;
struct node { Card val; link next; };

struct cardlist {
    link head, tail;
    int N;
};

struct sol {
    CardLIST lists[2];
};

// le funzioni sulle liste sono praticamente copie di quelle standard
// si riporta qui solo la SOLinit

SOL SOLinit(void) {
    SOL s = malloc(sizeof(*s));
    s->lists[0]=CardLISTinit();
    s->lists[1]=CardLISTinit();
    return s;
}
```

Problema di verifica

Sono dati $S1$ e $S2$, due sottoinsiemi non vuoti di S privi di intersezione (quindi $S1 \subset S$, $S2 \subset S$, e $S1 \cap S2 = \{\}$): in pratica si tratta di insiemi di carte estratte in sequenza da S , prima $S1$, poi $S2$ (preso dalla carte rimaste dopo aver tolto $S1$).

Si scriva una funzione che, dati $S1$ e $S2$, verifichi che per ogni carta di uno dei due insiemi esista almeno una carta dello stesso tipo/valore (ma di seme diverso) nell'altro insieme.

La funzione abbia prototipo

```
int checkCardLists(CardLIST S1, CardLIST S2);
```

```
/* per evitare un costo quadratico, si scandiscono con costo lineare le due liste,
   contando i valori in due vettori di contatori (una matrice di due righe).
   A partire dai contatori, la verifica vera e propria viene effettuata nella
   funzione checkCardCounts. La funzione interna checkCardCounts verra' usata
   nella checkSol (problema di ottimizzazione).
*/

int checkCardCounts(int valCount[2][14]) {
    for (int val=1; val<=13; val++) {
        if (valCount[0][val]>0 && valCount[1][val]==0 ||
            valCount[0][val]==0 && valCount[1][val]>0)
            return 0;
    }
    return 1;
}

int checkCardLists(CardLIST S1, CardLIST S2) {
    int valCount[2][14]={0};
    link x;
    for (x=S1->head; x!=NULL; x=x->next)
        valCount[0][x->val.val]++;
    for (x=S2->head; x!=NULL; x=x->next)
        valCount[1][x->val.val]++;
    return checkCardCounts(valCount);
}
```

Problema di ricerca e ottimizzazione

Dall'insieme S si vogliono estrarre due sequenze (catene ordinate di carte, anche queste prive di carte in comune, non necessariamente utilizzando tutte le carte in S) della stessa lunghezza, tali da rispettare i vincoli seguenti:

- A. in ognuna delle due sequenze, due carte consecutive devono
 - essere di seme diverso
 - avere valore che differisce al più di 2
 - le figure devono essere almeno la metà dei numeri
- B. Gli insiemi della carte nelle due sequenze devono rispettare il criterio del problema precedente: per ogni carta di una delle sequenze deve esistere almeno una carta dello stesso tipo/valore (ma di seme diverso) nell'altra sequenza.

Si scriva una funzione che, dato l'insieme S , determini la lunghezza massima che possono avere le due sequenze, rispettando i vincoli sopra elencati. La funzione abbia prototipo

```
SOL bestSequences(CardLIST S);
```

```
/* Si cercano due sequenza della stessa lunghezza, per cui la struttura dati piu semplice
   (e piu' vicina a quanto fatto nel corso) per gestire la soluzione e' un vettore in cui
   rappresentare una sola permutazione delle carte disponibili: la prima sequenza corrisponde
   agli indici pari (0,2,4,...), la seconda agli indici dispari (1,3,5,...).
   Il modello più adatto sono le permutazioni semplici (oppure le disposizioni, a patto di
   utilizzarle correttamente) in quanto occorre provare tutti i possibili ordini per le carte
   selezionate.
   NON conviene decisamente utilizzare liste, ne' per l'insieme di carte disponibili ne' per la
   soluzione, all'interno della funzione ricorsiva, in quanto non permettono accesso diretto.
   La lista in ingresso viene quindi convertita in un vettore (cards). La soluzione viene
   gestita con vettori. Solo al termine si converte la soluzione in due liste.
*/
```

```

void perm_sempl(int pos, int mark[], Card cards[],
               int sol[], int bestSol[], int *bestNumP,
               int nFig[2], int n, int nFigTot, int valCount[2][14]);

SOL bestSequences(CardLIST S) {
    int n=S->N, bestNum=0, nFig[2]={0,0}, nFigTot=0;
    Card *cards = malloc(n*sizeof(Card));
    int *mark = calloc(n,sizeof(int));
    int *sol = malloc(n*sizeof(int));
    int *bestSol = malloc(n*sizeof(int));
    int valCount[2][14]={0};
    /* passa da lista a vettore */
    link x;
    int i;
    for (i=0, x=S->head; x!=NULL; i++, x=x->next) {
        cards[i]=x->val;
        if (cards[i].val>10) nFigTot++;
    }

    perm_sempl(0, mark, cards, sol, bestSol, &bestNum, nFig, n, nFigTot, valCount);

    /* genera soluzione */
    SOL sollist = SOLinit();
    for (int i=0; i<bestNum; i++)
        CardLISTinsTail (sollist->lists[i%2], cards[bestSol[i]]);

    free(cards);
    free(mark);
    free(sol);
    free(bestSol);
    return sollist;
}

/* verifica per prima cosa i primi due vincoli. Poi si verifica non il terzo vincolo,
   ma la possibilita' che il caso migliore (se si usassero tutte le figure presenti) possa
   soddisfare il vincolo. Si potrebbe ulteriormente usare il terzo vincolo con un paradigma
   che (pur se assimilabile a pruning) si chiama "branch and bound": consiste nell'evitare
   una ricorsione se non ha speranza di migliorare l'ottimo attuale; in base al numero
   di figure ancora disponibili si potrebbe stimare la lunghezza massima raggiungibile,
   verificando la possibilita' di migliorare l'ottimo.
   Il paradigma branch and bound non fa parte del programma di Algoritmi.
*/
int prune(int sol[], Card cards[], int pos, int i, int nFig[2], int n, int nFigTot) {
    if (pos<2) return 0; // niente da verificare
    int prev=sol[pos-2];
    if (abs(cards[i].val-cards[prev].val)>2 ||
        cards[i].seed == cards[prev].seed)
        return 1;
    int nNum = pos-(nFig[0]+nFig[1]); // carte numeriche gia' usate
    if (2*nFigTot < (nNum-1)) // non ci sono abbastanza figure per raggiungere i numeri
        return 1;
    return 0;
}

int checkSol(int nFig[2], int pos, int valCount[2][14]) {
    if (3*nFig[0]<pos/2 || 3*nFig[0]<pos/2) // le sequenze hanno lunghezza pos/2
        return 0;
    return checkCardCounts(valCount);
}

void updateBestSol(int sol[], int pos, int bestSol[], int *bestNumP) {
    *bestNumP = pos;
    for (int i=0; i<pos; i++)
        bestSol[i]=sol[i];
}

```

```

/* modello delle permutazioni semplici adattato al problema:
   il pruning verifica le prime due condizioni.
   Si verifica l'accettabilit  di una soluzione ad ogni ricorsione pari.
*/
void perm_sempl(int pos, int mark[], Card cards[],
               int sol[], int bestSol[], int *bestNumP,
               int nFig[2], int n, int nFigTot, int valCount[2][14]) {
    int i;
    if (pos%2==0) {
        /* numero di carte pari; verifica soluzione */
        if (pos > *bestNumP && checkSol(nFig,pos,valCount))
            updateBestSol(sol, pos, bestSol, bestNumP);
    }
    if (pos >= n) {
        /* carte finite: soluzione gia' gestita */
        return;
    }

    for (i=0; i<n; i++)
        if (!mark[i]) {
            if (!prune(sol,cards,pos,i,nFig,n,nFigTot)) {
                mark[i] = 1;
                sol[pos] = i;
                valCount[pos%2][cards[i].val]++; // conta le carte di questo valore
                if (cards[i].val>10) // figura
                    nFig[pos%2]++; // conta figure (evita costo 0(pos) in checkSol)
                perm_sempl(pos+1, mark, cards, sol, bestSol, bestNumP, nFig,
                           n, nFigTot, valCount);
                valCount[pos%2][cards[i].val]--;
                if (cards[i].val>10) // figura
                    nFig[pos%2]--;
                mark[i] = 0;
            }
        }
}

```

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti):

- indicare nell'elaborato e nella relazione nome, cognome e numero di matricola.
- se non indicato diversamente,   consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne. Allegare gli header file oppure scrivere le definizioni dei tipi e i prototipi delle funzioni usate.
- consegna delle relazioni (per entrambe le tipologie di prova di programmazione): entro il 23/02/2026, alle ore 23:59, mediante caricamento su Portale. Le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale). **QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERR  APPLICATA UNA PENALIZZAZIONE.** Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto in aula. Non verranno corretti i compiti di cui non sar  stata inviata la relazione nei tempi stabiliti.